Python Libraries

NUMPY

Numpy

Introduction to Numpy

- N-dim array for fast mathematical calculation
- Homogeneous data structure
- Written in C/C++
- Python List is generalized and thus slow
- Import modules: `import numpy as np`

Numpy

Initialization

- Size is important parameter for all of them
- Zeros-initialized with zero values
- Ones-initialized with one values
- Empty-uninitialized array (Contains garbage value)
- Full-initialized with value passed
- Random-different methods to initialize values
- Linspace-generate equally spaced numbers between start and stop values
- …

Numpy

Initialization

```
arr = np.empty(shape=(2,2))
print(arr)

arr = np.zeros(shape=(2, 2))
print(arr)

arr = np.arange(start=2, stop=10, step=1)
print(arr)
```

Numpy

Initialization

```
arr = np.linspace(start=0, stop=10, num=10)
print(arr)
```

```
arr = np.random.randint(low=0, high=10, size=(2, 2))
print(arr)
```

Numpy

Access

- Very similar to that of accessing list

```
arr[:,:]
```

```
array([[8, 4, 0],
       [2, 8, 5],
       [4, 4, 3]])
```

```
arr[:,0:2]
```

```
array([[8, 4],
       [2, 8],
       [4, 4]])
```

```
arr[:, 2]
```

```
array([0, 5, 3])
```

```
arr[0:2, :2]
```

```
array([[8, 4],
       [2, 8]])
```

Numpy

Concatenation

- Axis = 0 means vertical
- Axis =1 mean horizontal
- Concatenate-join numpy array along provided axis
- Hstack-join numpy array horizontally
- Vstack-joining numpy array vertically

Numpy

Concatenation

```
a
```

```
array([[4, 4, 8],
       [3, 8, 9],
       [6, 3, 2]])
```

```
b
```

```
array([[0, 7, 1],
       [0, 6, 9],
       [9, 0, 2]])
```

```
np.vstack([a, b])
```

```
array([[4, 4, 8],
       [3, 8, 9],
       [6, 3, 2],
       [0, 7, 1],
       [0, 6, 9],
       [9, 0, 2]])
```

```
np.hstack([a, b])
```

```
array([[4, 4, 8, 0, 7, 1],
       [3, 8, 9, 0, 6, 9],
       [6, 3, 2, 9, 0, 2]])
```

Numpy

Concatenation

```
arr1 = np.array([1,2,3])
arr2 = np.array([4,5,6])
arr3 = np.array([7,8,9])
```

```
np.concatenate((arr1, arr2, arr3), axis=0)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
arr1 = np.array([[1,2,3], [3,4,5]])
arr2 = np.array([[7,8,9], [10,11,12]])
```

```
np.concatenate((arr1, arr2), axis=1)
```

```
array([[ 1,  2,  3,  7,  8,  9],
       [ 3,  4,  5, 10, 11, 12]])
```

Numpy

Splitting

- Splits array into subarrays
- Split-Splits the array as per axis mentioned
- Hsplit – This splits the horizontal axis
- Vsplit – This splits the vertical axis

Numpy

Splitting

```
a
```

```
array([[4, 4, 8],
       [3, 8, 9],
       [6, 3, 2]])
```

```
a1, a2, a3 = np.split(a, [1, 2], axis=0)
print(a1, a2, a3)
```

```
[[4 4 8]] [[3 8 9]] [[6 3 2]]
```

Numpy

Array attributes

```
a
```

```
array([[4, 4, 8],
       [3, 8, 9],
       [6, 3, 2]])
```

```
a.size
```

```
9
```

```
a.shape
```

```
(3, 3)
```

```
a.ndim
```

```
2
```

```
a.max(), a.min()
```

```
(9, 2)
```

Numpy

Reshaping

- Learning algorithms expects data in certain shape and dimension
- Using reshaping utility we can convert data into desired shape
- But, the desired transformation will also be of same size
- Add 1 dimensions will not alter data size

Numpy

Reshaping

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[5, 3, 6, 6, 4, 9],
       [0, 0, 7, 3, 1, 9],
       [7, 2, 7, 4, 4, 4],
       [3, 5, 0, 1, 5, 8]])
```

```
a = a.reshape(3, 8)
a
```

```
array([[5, 3, 6, 6, 4, 9, 0, 0],
       [7, 3, 1, 9, 7, 2, 7, 4],
       [4, 4, 3, 5, 0, 1, 5, 8]])
```

Numpy

Adding dimension

- Learning algorithms consumes data in two dimension
- Below, we are converting 2-D array to 3-D

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[8, 9, 2, 1, 6, 7],
       [9, 6, 8, 0, 6, 3],
       [2, 9, 4, 6, 2, 8],
       [4, 0, 2, 1, 8, 4]])
```

```
a1 = a.reshape(3, 2, -1)
a1.shape
```

```
(3, 2, 4)
```

```
a2 = a[:,:, np.newaxis]
a2.shape
```

```
(4, 6, 1)
```

Numpy

Transpose

- Generates the transposition of an array using the function ***np.transpose***
- Does not affect the original array, but it will create a new array

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[9, 5, 0, 1, 0, 2],
       [3, 3, 4, 9, 0, 2],
       [8, 3, 8, 2, 5, 9],
       [9, 4, 8, 4, 4, 6]])
```

```
a.T
```

```
array([[9, 3, 8, 9],
       [5, 3, 3, 4],
       [0, 4, 8, 8],
       [1, 9, 2, 4],
       [0, 0, 5, 4],
       [2, 2, 9, 6]])
```

```
np.transpose(a)
```

```
array([[9, 3, 8, 9],
       [5, 3, 3, 4],
       [0, 4, 8, 8],
       [1, 9, 2, 4],
       [0, 0, 5, 4],
       [2, 2, 9, 6]])
```

Numpy

Flatten

- Flatten creates a copy ò the input array flattened to one dimension
- Does not affect the original array, but it will create a new array

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[9, 5, 0, 1, 0, 2],
       [3, 3, 4, 9, 0, 2],
       [8, 3, 8, 2, 5, 9],
       [9, 4, 8, 4, 4, 6]])
```

```
a.flatten()
```

```
array([9, 5, 0, 1, 0, 2, 3, 3, 4, 9, 0, 2, 8, 3, 8, 2, 5, 9, 9, 4, 8, 4,
       4, 6])
```

Numpy

Statistical Functions

- Numpy has a huge list of mathematical functions make implementing machine learning algorithms very easy
- A few of them are
  - Max
  - Min
  - Mean
  - Std
  - Cov

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[9, 5, 0, 1, 0, 2],
       [3, 3, 4, 9, 0, 2],
       [8, 3, 8, 2, 5, 9],
       [9, 4, 8, 4, 4, 6]])
```

```
np.mean(a)
```

```
4.5
```

```
np.median(a)
```

```
4.0
```

```
np.std(a)
```

```
3.0276503540974917
```

Numpy

Statistical Functions

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[9, 5, 0, 1, 0, 2],
       [3, 3, 4, 9, 0, 2],
       [8, 3, 8, 2, 5, 9],
       [9, 4, 8, 4, 4, 6]])
```

```
np.cov(a)
```

```
array([[12.56666667, -0.9       ,  1.56666667,  3.56666667],
       [-0.9       ,  9.1       , -4.3       , -0.7       ],
       [ 1.56666667, -4.3       ,  8.56666667,  5.16666667],
       [ 3.56666667, -0.7       ,  5.16666667,  4.96666667]])
```

Numpy

Statistical Functions

```
a = np.random.randint(low=0, high=10, size=(4, 6))
a
```

```
array([[9, 5, 0, 1, 0, 2],
       [3, 3, 4, 9, 0, 2],
       [8, 3, 8, 2, 5, 9],
       [9, 4, 8, 4, 4, 6]])
```

```
np.quantile(a,0.25)
```

```
2.0
```

Numpy

Broadcasting

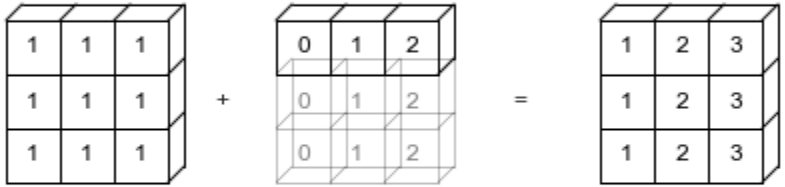
- Single row matrix are known as vectors
- Broadcasting is a technique using which Numpy dose mathematical computation on data of different shape and dimension
- We might need to reshape the data sometimes to enable broadcasting

Numpy

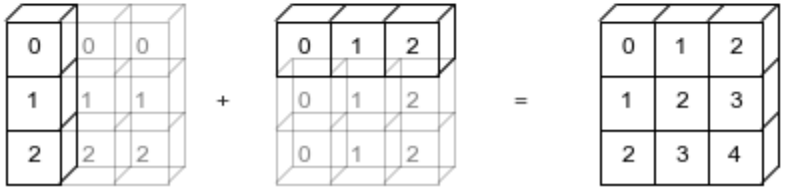
Broadcasting

$\texttt{np.arange}(3) + 5$

$\texttt{np.ones}((3,3)) + \texttt{np.arange}(3)$



$\texttt{np.arange}(3).\texttt{reshape}((3,1)) + \texttt{np.arange}(3)$



Demonstration

Numpy Practice

Thank you